

DPP Assignment 4

Simon Vinding Brodersen(rzn875)

December 17, 2025

1 Apply Reverse AD to a *map - reduce min* Composition

My code validates and is provided below:

```
def task_mapomin [n] (as: [n]f32) (y_bar: f32) : (f32, [n]f32) =
  let bs = map (\a -> let b = a * a - 0.5 * a in b) as
  let (imin, y) =
    zip (iota n) bs
    |> reduce_comm (\(i1, b1) (i2, b2) ->
      if b1 == b2
      then if i1 < i2
        then (i1, b1)
        else (i2, b2)
      else if b1 < b2
        then (i1, b1)
        else (i2, b2))
      (n, f32.highest)
  let bs_bar = tabulate n (\i -> if i == imin then y_bar else 0.0f32)
  let as_bar =
    map2 (\a b_bar ->
      let a_bar = (2 * a - 0.5) * b_bar
      in a_bar)
    as
    bs_bar
  in (y, as_bar)
```

The lifted operator is what is the function passed to the `reduce_comm` function. This is both associative and cumulative, as I do tie breaks on the indices. Essentially this works like the regular minimum function, but breaking ties by choosing the smaller of the two indices. This means that no matter what order you give the inputs, if the inputs are equivalent, then the result is the same.

First, then primal is called, where we have substituted the `reduce_comm` call with the one including the lifted operator mentioned before. Next, we have the `tabulate` call, which is what corresponds to the adjoint of the `reduce_comm`. This `tabulate` also populates the $\bar{a}$ s, which previously is not used and as such is initialized to 0. The last map, then corresponds to the adjoint of the map.

Table 1: Benchmark runtimes and speedups for MapOmin.

Method	Runtime (μ s)		Overhead from primal	
	10M elements	100M elements	10M elements	100M elements
pomin_primal	67	303	-	-
vjp2	157	910	2.34	3.00
manual	147	1029	2.19	3.40

The runtimes can be seen in Table 1. Here we see that in general the run times of my implementation is very close to that of the built-in vjp2, which assuming it also does the same optimisation is to be expected.

2 Generic Reverse-AD of Reduce with Invertible Operator

My code validates and is provided below:

```
def vjp2_red_inv 't 't_lft [n]
  (op : t -> t -> t)
  (op_lft : t_lft -> t_lft -> t_lft)
  (ne: t_lft)
  (cfwd : t -> t_lft)
  (cbwd : t_lft -> t)
  (op_inv : t_lft -> t -> t)
  (as : [n]t)
  (rs_bar: t)
  : (t, [n]t) =
-- please replace the dummy code below with your correct implementation
let t_prim = map cfwd as |> reduce_comm (op_lft) ne
let y = cbwd t_prim
let as_bar = map (\a ->
  let p' = op_inv t_prim a
  in vjp (\x -> op x p') a rs_bar
) as
in (y, as_bar)
```

If we assume no special cases, then the differentiation of an arbitrary reduce operation would lead to two scans and a map call. We are able to circumvent this, such that we instead have a single map call, and then whatever is the differentiation of the operator, which often is just constant time. This means, that we are able to go from work $O(n)$ and span $O(\lg n)$ to work $O(n)$ and span $O(1)$ and we should also have reduced the constant factor by now only doing a single map call.

Table 2: Benchmark runtimes and speedups for Mul and Sop groups.

Operator	Method	Runtime (μ s)		Overhead from primal	
		50M elements	500M elements	50M elements	500M elements
Mul	primal	296	2734	-	-
	ppad	3414	33662	11.53	12.31
	vjp2	901	8603	3.04	3.15
	manual	920	8717	3.11	3.19
Sop	primal	594	-	-	-
	ppad	6214	-	10.46	-
	vjp2	3864	-	6.51	-
	manual	1450	-	2.44	-

The runtimes can be seen in Table 2, which shows that in the Mul case the runtimes of vjp2 and manual are almost equivalent. However, in the Sop case the manual implementation is significantly faster. This is most likely because futhark has this optimisation built in for the built in operators such as multiplication, but not for the sop case.

3 Generic Reverse-AD of Reduce-By-Index with Invertible Operator

My code validates and is provided below:

```
def vjp_redbyind_inv 't 't_lft [m] [n]
  (op: t -> t -> t)
  (op_lft: t_lft -> t_lft -> t_lft)
  (ne: t_lft)
  (cfwd: t -> t_lft)
  (cbwd: t_lft -> t)
  (op_inv: t_lft -> t -> t)
  -- \^ function params
  (dst: [m]t)
  (ks: [n]i64)
  (vs: [n]t)
  (rs_bar: [m]t) : ([m]t, [m]t, [n]t) =
let hsL = map cfwd vs |> hist op_lft ne m ks
let hs = map cbwd hsL
let rs = map2 op dst hs
let g d h r_bar = vjp (\x -> op x d) h r_bar
let hs_bar = map3 g dst hs rs_bar
let dst_bar = map3 g hs dst rs_bar
let f k v = let p' = op_inv hsL[k] v in vjp (\x -> op x p') v hs_bar[k]
let vs_bar = map2 f ks vs
in (rs, dst_bar, vs_bar)
```

In the general case of inverting the `reduce_by_idx` we would need multiple scans. Hinted to via the slides, this could be done by sorting the key-value pairs according to the keys, but still would be a very expensive operation. In this special case, we are able to completely circumvent this and

simply use maps only (together with the operator differentiated and inverse). This leads to an order of magnitudes faster execution than that of the none specialised method.

Table 3: Benchmark runtimes and overheads for Mul and Sop (10M elements).

Operator	Method	Runtime (μ s)			Overhead from primal		
		31 bins	401 bins	50k bins	31 bins	401 bins	50k bins
Mul	primal	231	228	621	-	-	-
	vjp2	770	632	2143	3.33	2.77	3.45
	vjp2_inv	569	478	1060	2.46	2.10	1.71
Sop	primal	344	329	2145	-	-	-
	vjp2	9986	13133	20682	29.03	39.92	9.64
	vjp2_inv	579	591	2597	1.68	1.80	1.21

The benchmarking results can be seen in Table 3. Again, we see that the Mul has quite similar runtimes but in the non-builtin operator sop, there is a significant difference in the runtimes.

4 Implement kMeans with Nested Forward & Reverse AD

My code validates and is provided below:

```
-- Task 4 (a)
def cost [n] [k] [d] (points: [n][d]f32) (centres: [k][d]f32) : f32 =
  let min_distances =
    map (\p ->
      map (\c -> euclid_dist_2 p c) centres
      |> reduce f32.min f32.inf)
    points
  in reduce (+) 0f32 min_distances

def tolerance = 1 : f32

-- Task 4 (b)
let (cost', cost'') =
  let f centers' = vjp (cost points) centers' 1f32
  in jvp2 f cluster_centers (replicate (k * d) 1f32 |> unflatten)

-- Task 4 (c)
let new_centers =
  let f i j =
    let tmp = cost'[i, j]
    in if tmp == 0f32
      then cluster_centers[i, j]
      else cluster_centers[i
        ,j]
    - (cost'[i, j] / cost'[i, j])
  in map (\i -> map (f i) (iota d)) (iota k)
```

Task a: The cost function simply nests a map inside a map finding the distance from each point to each center, and then finding the minimum. Once this is done, we simply reduce over all of the

minimum distances to find the cost. The `euclid_dist_2` function was provided to us.

Task b: Here I simply followed the comments explanation. I.e. I apply the `vjp` function to the (cost points) in the point that corresponds to the current cluster center, which is passed to the function `f` and simply adjoint with 1. `Jvp2` is then applied to this function, which receives the cluster centers and applies them together with the matrix of ones.

Task c: This task again follows the comment, but builds upon this with checking for division by 0. In such a case, we do not update the value of the center, and in all other cases, we simply provide what the comment states. This is then mapped to all `i` and `j` combinations.